

Exercício 12

Integrante: Wesley Bernardes (020321)

1) Faça um procedimento para mostrar o nome do Delivery, o nome do cliente e o descritivo da entrega, envolvendo Clientes, Delivery e Controle_Entrega do banco supermercado. Em seguida, chame o procedimento criado. Lembre-se da sintaxe:

```
CREATE OR ALTER PROCEDURE [dbo].[_sp_mostrar_entregas]
AS
BEGIN
    SELECT
        d.nome_moto AS nome_entregador,
        c.nome AS nome_cliente,
        ce.descritivo AS descritivo

    FROM controle_entregas AS ce

    INNER JOIN deliveries AS d
        ON ce.cod_entregador = d.cod_entregador

    INNER JOIN clientes AS c
        ON ce.cod_cliente = c.cod_cliente;
END

EXEC _sp_mostrar_entregas
```

Executando a procedure o resultado conforme meu banco populado foi:

Results		Messages	
	nome_entregador	nome_cliente	descricao
1	Speed Racer	Ana Souza	NULL
2	Vento Forte	Ana Souza	NULL
3	Relâmpago Azul	Ana Souza	NULL

2) Considerando o MER ao lado, faça um procedimento cadastrar novos fornecedores. Em seguida teste o procedimento fazendo uma nova inserção. Faça uma consulta para verificar se os dados foram inseridos corretamente.

```

CREATE OR ALTER PROCEDURE [dbo].[_sp_cadastrar_fornecedor]
    @p_cnpj    VARCHAR(18),
    @p_nome    VARCHAR(50),
    @p_endereco VARCHAR(30),
    @p_tipo    INT
AS
BEGIN
    INSERT INTO fornecedores (cnpj, nome, endereco, tipo)
    VALUES (@p_cnpj, @p_nome, @p_endereco, @p_tipo);
END

EXEC _sp_cadastrar_fornecedor
    @p_cnpj    = '12.345.678/0001-80',
    @p_nome    = 'Fornecedor novo',
    @p_endereco = 'Rua Novo Fornecedor, 100',
    @p_tipo    = 3;

SELECT * FROM fornecedores;

```

Executando a procedure o resultado conforme meu banco populado foi:

Results		Messages			
	cnpj	nome	endereco	tipo	criado_em
1	01.234.567/0001-89	Fornecedor Exemplo	Rua das Flores, 123	1	2025-05-21 20:33:34.637
2	12.345.678/0001-80	Fornecedor novo	Rua Novo Fornecedor, 100	3	2025-05-23 22:49:13.710
3	12.345.678/0001-99	Fornecedor Novo	Rua Inovação, 100	3	2025-05-21 21:49:41.123
4	22.333.444/0001-55	Fornecedor X	Av. Central, 200	1	2025-05-21 21:49:52.640
5	98.765.432/0001-10	Alimentos e Bebidas LTDA	Av. Brasil, 456	2	2025-05-21 20:33:34.637

3) Refaça o exercício anterior para além de inserir um fornecedor mostrar os fornecedores que já foram inseridos, ou seja, o mesmo procedimento terá duas funções, inserir e exibir aquilo que já existe na base.

```
CREATE OR ALTER PROCEDURE [dbo].[_sp_cadastrar_e_listar_fornecedores]
s]
    @p_cnpj    VARCHAR(18),
    @p_nome    VARCHAR(50),
    @p_endereco VARCHAR(30),
    @p_tipo    INT
AS
BEGIN
    INSERT INTO fornecedores (cnpj, nome, endereco, tipo)
    VALUES (@p_cnpj, @p_nome, @p_endereco, @p_tipo);

    SELECT * FROM fornecedores;
END

EXEC _sp_cadastrar_e_listar_fornecedores
    @p_cnpj    = '22.333.444/0001-99',
    @p_nome    = 'Fornecedor do cadastrar e listar',
    @p_endereco = 'Av. Central, 200',
```

```
@p_tipo = 1;  
GO
```

Executando a procedure o resultado conforme meu banco populado foi:

	cnpi	nome	endereço	tipo	criado_em
1	01.234.567/0001-89	Fornecedor Exemplo	Rua das Flores, 123	1	2025-05-21 20:33:34.637
2	12.345.678/0001-80	Fornecedor novo	Rua Novo Fornecedor, 100	3	2025-05-23 22:49:13.710
3	12.345.678/0001-99	Fornecedor Novo	Rua Inovação, 100	3	2025-05-21 21:49:41.123
4	22.333.444/0001-55	Fornecedor X	Av. Central, 200	1	2025-05-21 21:49:52.640
5	22.333.444/0001-99	Fornecedor do cadastrar e listar	Av. Central, 200	1	2025-05-23 22:52:15.140
6	98.765.432/0001-10	Alimentos e Bebidas LTDA	Av. Brasil, 456	2	2025-05-21 20:33:34.637

4) Faça um procedimento para retornar para uma variável de ambiente a idade da conta mais velha e nome do dono dela. Lembre-se limitar o resultado da consulta aninhada (Limit) a 1. Depois mostre os resultados usando variável de ambiente.

```
CREATE OR ALTER PROCEDURE [dbo].[_sp_idade_conta_mais_velha]  
    @p_idade INT OUTPUT,  
    @p_nome_dono VARCHAR(30) OUTPUT  
AS  
BEGIN  
    SELECT TOP 1  
        @p_idade = DATEDIFF(YEAR, desde, GETDATE()),  
        @p_nome_dono = nome  
    FROM clientes  
    ORDER BY desde ASC;  
END  
  
DECLARE @idade INT, @nome_dono VARCHAR(30);  
  
EXEC _sp_idade_conta_mais_velha  
    @p_idade = @idade OUTPUT,  
    @p_nome_dono = @nome_dono OUTPUT;
```

```

SELECT
    @idade AS idade_do_cliente_mais_antigo,
    @nome_dono AS nome_do_cliente;
GO

```

Executando a procedure o resultado conforme meu banco populado foi:

	idade_do_cliente_mais_antigo	nome_do_cliente
1	10	Ana Souza

5) Faça um procedimento para somar um valor decimal de entrada com uma variável qualquer. O programa deve usar SET, INOUT e DECLARE.

```

ALTER OR CREATE PROCEDURE _sp_somar_decimal
    @p_entrada DECIMAL(10,2),
    @p_acumulador DECIMAL(10,2)
AS
BEGIN
    DECLARE @soma DECIMAL(10,2);
    SET @soma = @p_acumulador + @p_entrada;
    SET @p_acumulador = @soma;
END;
GO

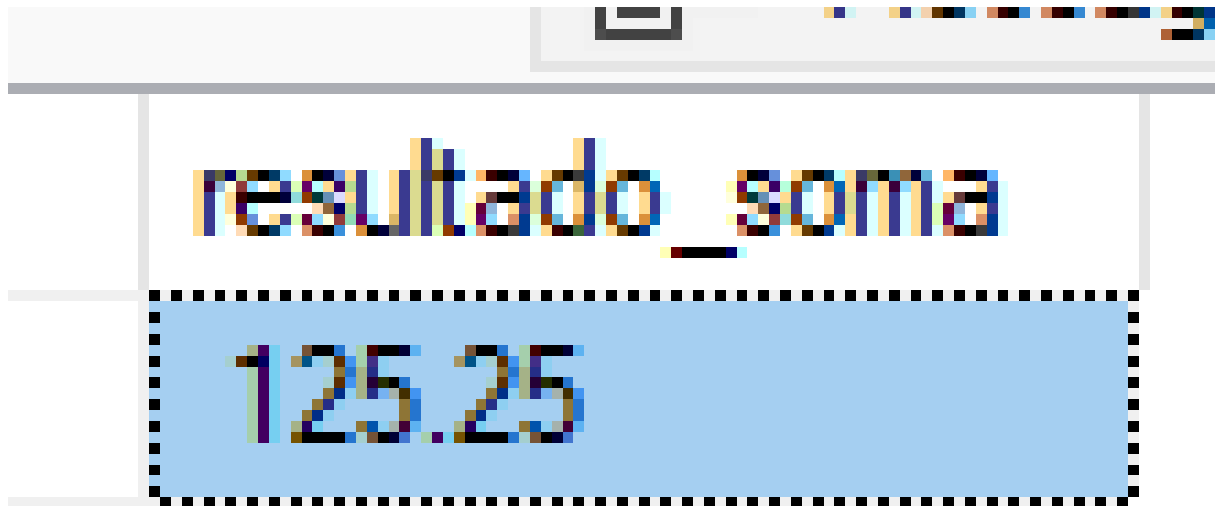
DECLARE @total DECIMAL(10,2) = 100.00;

EXEC _sp_somar_decimal
    @p_entrada = 25.25,
    @p_acumulador = @total OUTPUT;

```

```
SELECT @total AS resultado_soma;  
GO
```

Executando a procedure o resultado conforme meu banco populado foi:



resultado_soma
125.25

6) Faça um procedimento para multiplicar um valor inteiro de entrada com uma variável qualquer, em seguida esse valor deve ser inserido em algumas das tabelas, você pode também atualizar umas das tabelas com esse valor, ao invés de inserir. Por exemplo, esse valor pode ser um novo estoque de produto.

```
ALTER OR CREATE PROCEDURE [dbo].[_sp_multiplicar_e_atualizar_estoque]  
e]  
    @p_multiplicando INT,  
    @p_cod_produto INT  
AS  
BEGIN  
    DECLARE @novo_estoque INT;  
    SET @novo_estoque = @p_multiplicando * 2;
```

```

UPDATE produtos
SET estoque = @novo_estoque
WHERE cod_produto = @p_cod_produto;
END;

SELECT cod_produto, nome, estoque FROM produtos WHERE cod_produto
= 1;

EXEC _sp_multiplicar_e_atualizar_estoque
    @p_multiplicando = 5,
    @p_cod_produto = 1;

SELECT cod_produto, nome, estoque FROM produtos WHERE cod_produto
= 1;
GO

```

Executando a procedure o resultado conforme meu banco populado foi:

Results		Messages	
	cod_produto	nome	estoque
1	1	Arroz	100

	cod_produto	nome	estoque
1	1	Arroz	10

7) Remova uma quantidade qualquer de produtos da Tabela produtos, onde o nome

do produto comece com "a". Mostre o antes e o depois da remoção

```
ALTER OR CREATE PROCEDURE _sp_remove_produtos_a
    @p_quantidade INT
AS
BEGIN
    SELECT
        'ANTES' AS momento,
        cod_produto, nome, preco, estoque
    FROM produtos
    WHERE LOWER(nome) LIKE 'a%';

    UPDATE produtos SET estoque = estoque - @p_quantidade
    WHERE LOWER(nome) LIKE 'a%';

    SELECT
        'DEPOIS' AS momento,
        cod_produto, nome, preco, estoque
    FROM produtos
    WHERE LOWER(nome) LIKE 'a%';
END
GO

EXEC _sp_remove_produtos_a @p_quantidade = 10;
GO
```

Executando a procedure o resultado conforme meu banco populado foi:

	momento	cod_produto	nome	preco	estoque
1	ANTES	1	Arroz	5.99	10
2	ANTES	3	Açúcar	4.50	120

	momento	cod_produto	nome	preco	estoque
1	DEPOIS	1	Arroz	5.99	0
2	DEPOIS	3	Açúcar	4.50	110

8) Implemente o código ao lado e comente as linhas entre BEGIN e END. Os comentários devem explicar o que código faz.

```

ALTER OR CREATE PROCEDURE [dbo].[_sp_receita_geral]
    @total DECIMAL(18,2) OUTPUT
AS
BEGIN
    -- contador: itera sobre cada produto (inicia em 1)
    DECLARE @contador INT = 1;
    -- estoque atual do produto
    DECLARE @estoque INT;
    -- preço unitário do produto
    DECLARE @valor_unit DECIMAL(10,2);
    -- total de produtos na tabela
    DECLARE @total_itens INT;
    -- acumulador do valor total de todas as vendas
    DECLARE @soma DECIMAL(18,2) = 0;

    -- obtém a quantidade de registros em produtos
    SET @total_itens = (SELECT COUNT(*) FROM produtos);

    -- enquanto não percorrer todos os produtos
    WHILE @contador <= @total_itens

```

```

BEGIN
    -- busca estoque do produto de código = contador
    SET @estoque = (
        SELECT estoque
        FROM produtos
        WHERE cod_produto = @contador
    );
    -- busca preço unitário do mesmo produto
    SET @valor_unit = (
        SELECT preco
        FROM produtos
        WHERE cod_produto = @contador
    );
    -- acumula: soma += estoque * preço
    SET @soma = @soma + (@estoque * @valor_unit);
    -- próximo produto
    SET @contador = @contador + 1;
END

-- retorna o valor total
SET @total = @soma;
END;
GO

DECLARE @receita DECIMAL(18,2);
EXEC _sp_receita_geral @receita OUTPUT;
SELECT @receita AS receita_total;
GO

```

Executando a procedure o resultado conforme meu banco populado foi:

